

Topic 4: Informed Search and Exploration — সম্পূর্ণ Theory Guide

□ **Informed search uses problem-specific knowledge (heuristics) to guide the search towards the goal faster than uninformed search.**

১. Informed Search কী এবং কেন দরকার?

Uninformed search (যেমন BFS, DFS) problem সম্পর্কে কোনো extra knowledge ছাড়াই blindly সব node explore করে। কিন্তু **Informed Search** problem-specific knowledge ব্যবহার করে — এই knowledge কে বলা হয় **heuristic**।

সহজ কথায়: তুমি যদি Dhaka থেকে Chittagong যেতে চাও, তাহলে map না দেখে random রাস্তায় গেলে সেটা uninformed। কিন্তু যদি তুমি জানো "দক্ষিণ-পূর্বে যেতে হবে" — এই জ্ঞানটাই heuristic, আর এটা ব্যবহার করাই informed search।

Informed Search (Two Main Branches)

- Under the umbrella of Informed Search
 - Heuristic Tree Search
 - Heuristic Local Search
- Informed search includes both goal-directed tree searches and optimization-oriented local searches, all using heuristics to improve efficiency.

Heuristic Tree Search

- Path-finding / Goal search
 - Greedy Best-First Search
 - A*
 - Recursive Best-First Search (RBFS)
 - Memory-bounded A*

Heuristic Local Search

- Optimization (Find best solution)
 - Hill-climbing
 - Simulated annealing
 - Local Beam search
 - Genetic algorithms
- Application: optimization, scheduling, machine learning hyperparameter tuning, game AI, design problems.

২. Heuristic Function $h(n)$ — Theory

Dictionary definition অনুযায়ী heuristic হলো: *"A rule of thumb, simplification, or educated guess that reduces or limits the search for solutions in domains that are difficult and poorly understood."*

Formally:

$h(n)$ = estimated cost of the cheapest path from node n to the goal node

এটা একটা **function** যেটা বলে — "আমি এখন যে node-এ আছি, সেখান থেকে goal পর্যন্ত যেতে কত খরচ লাগতে পারে।" এটা exact নয়, **estimate**।

চারটি **property** মনে রাখো:

Rule of Thumb: কোনো exact formula নয়, practical guideline। উদাহরণ — হারিয়ে গেলে নদীর স্রোত ধরে নামো, সম্ভ্যতা পাবে।

Simplification: জটিল সমস্যাকে সহজ করে দেখা। উদাহরণ — মুদিখানায় কম ingredient-এর খাবার বেছে নেওয়া মানে স্বাস্থ্যকর।

Educated Guess: অভিজ্ঞতা-ভিত্তিক অনুমান, exact calculation নয়। উদাহরণ — PC slow হলে restart দিলে ঠিক হতে পারে।

Limits the Search: সব option না দেখে likely solution-এ focus করা। উদাহরণ — চাবির বান্ডিল থেকে সবচেয়ে worn-out চাবিটা আগে try করো।

Key point: Heuristic always perfect নয়, কিন্তু "good enough" হলেই হয়।

৩. Best-First Search — Framework

সব informed search-এর base হলো **Best-First Search**। এখানে প্রতিটা node-এর একটা **evaluation function $f(n)$** থাকে এবং সেই অনুযায়ী priority queue তে রাখা হয়। সবচেয়ে "ভালো" মনে হওয়া node আগে expand হয়।

৪. Greedy Best-First Search — Theory

Evaluation function: $f(n) = h(n)$

শুধু heuristic দেখে সিদ্ধান্ত নেয়। যে node-টা goal-এর সবচেয়ে কাছে মনে হয়, সেটাই expand করে। "কতটা পথ আসা হয়েছে" সেটা সম্পূর্ণ ignore করে।

সমস্যা কোথায়?

এটা **complete** নয় এবং **optimal** নয়। কারণ — শুধু $h(n)$ দেখে চললে false start হতে পারে। যেমন lasi থেকে Fagaras যেতে চাইলে greedy ভুল দিকে যেতে পারে কারণ $h(n)$ locally ভালো মনে হলেও globally ভুল।

Complexity: Worst case — $O(b^m)$, কিন্তু ভালো heuristic হলে অনেক কম। (m is maximum depth of search tree and b is branching factor)

Analogy: তুমি maze-এ আছো, শুধু সামনের দেওয়াল দেখে সিদ্ধান্ত নিচ্ছো — dead end-এ আটকে যেতে পারো।

৫. A* Search — সবচেয়ে গুরুত্বপূর্ণ Theory

৫.১ মূল সূত্র

$$f(n) = g(n) + h(n)$$

এখানে:

- $g(n)$ = start থেকে node n পর্যন্ত আসার **actual (exact) cost**
- $h(n)$ = node n থেকে goal পর্যন্ত **estimated cost** (heuristic)
- $f(n)$ = total estimated cost of the optimal path through n

A* এমন path avoid করে যেগুলো ইতিমধ্যে expensive। Greedy যেখানে শুধু "সামনে কতটুকু বাকি" দেখে, A* সেখানে "এখন পর্যন্ত কত লেগেছে + সামনে কত লাগতে পারে" দুটোই দেখে।

৫.২ Admissible Heuristic — অনেক গুরুত্বপূর্ণ Theory

A* optimal হওয়ার জন্য heuristic-কে **admissible** হতে হবে।

Definition: A heuristic $h(n)$ is admissible if it **never overestimates** the cost to reach the goal.

Formally: $h(n) \leq h^*(n)$ — যেখানে $h^*(n)$ হলো n থেকে goal পর্যন্ত **true/actual cost**।

Admissible heuristic সবসময় **optimistic** — সে কখনো বলে না "এর চেয়ে বেশি লাগবে" যদি আসলে কম লাগে।

উদাহরণ: **straight-line distance (hSLD)** সবসময় admissible কারণ সরলরেখার দূরত্ব কখনো রাস্তার দূরত্বের চেয়ে বেশি হয় না।

৫.৩ Optimality of A* — Proof এর Theory

এটা অনেক important। ধরো $G2$ একটা suboptimal goal node, আর C^* হলো optimal solution-এর cost।

যেহেতু $G2$ suboptimal: $g(G2) > C^*$ যেহেতু $G2$ একটা goal node: $h(G2) = 0$ তাই: $f(G2) = g(G2) + 0 = g(G2) > C^*$

এখন optimal path-এর উপর একটা node n নাও। যেহেতু $h(n)$ admissible: $f(n) = g(n) + h(n) \leq C^*$

তাহলে: $f(n) \leq C^* < f(G2)$

Conclusion: A* সবসময় $f(G2)$ এর আগে $f(n)$ expand করবে — তাই suboptimal goal কখনো return হবে না। এটাই A*-এর optimality-র proof।

৫.৪ A* Evaluation Summary

Complete: হ্যাঁ — যদি finite nodes থাকে যেখানে $f < f(G)$ ।

Optimal: হ্যাঁ — যদি $h(n)$ admissible হয়।

Time Complexity: Exponential — solution-এর length-এর সাথে exponentially বাড়ে।

Space Complexity: সব generated node memory-তে রাখে। A* সাধারণত time-এর আগে space শেষ করে ফেলে — এটা একটা বড় সীমাবদ্ধতা।

Optimally efficient: কোনো optimal algorithm A* থেকে কম node expand করতে পারে না (given same $h(n)$)।

৬. Heuristic Functions-এর তুলনা — 8-Puzzle Example

8-puzzle-এ দুটো বিখ্যাত heuristic:

h1 = Misplaced Tiles: যতটি tile তাদের goal position-এ নেই, সেই সংখ্যা। উদাহরণে $h1 = 8$ ।

h2 = Manhattan Distance: প্রতিটি tile-কে তার goal position-এ পৌঁছাতে কতটা horizontal + vertical move লাগবে তার যোগফল। উদাহরণে $h2 = 3+1+2+2+2+3+3+2 = 18$ ।

কোনটা ভালো? $h2$ কারণ $h2 \geq h1$ সবসময়। $h2$ হলো **more informed** — actual cost-এর আরও কাছাকাছি estimate দেয়, কিন্তু কখনো overestimate করে না। এটাকে বলা হয় $h2$ **dominates** $h1$ । যখন $h2$ dominates $h1$, তখন A^* দিয়ে $h2$ ব্যবহার করলে সবসময় কম বা সমান node expand হবে।

৭. Local Search Algorithms — Theory

এতক্ষণ যা দেখলে সেখানে **path** important ছিল। কিন্তু অনেক problem-এ path গুরুত্বপূর্ণ নয়, **final state**-ই solution। যেমন 8-queens-এ কুইনের arrangement দরকার, কীভাবে পৌঁছেছি সেটা দরকার নেই।

Local search-এর **advantages**:

- খুব কম memory লাগে — শুধু current state রাখে
- বিশাল বা infinite state space-এ কাজ করে
- Optimization problem-এ কার্যকর (সেরা state খোঁজা)

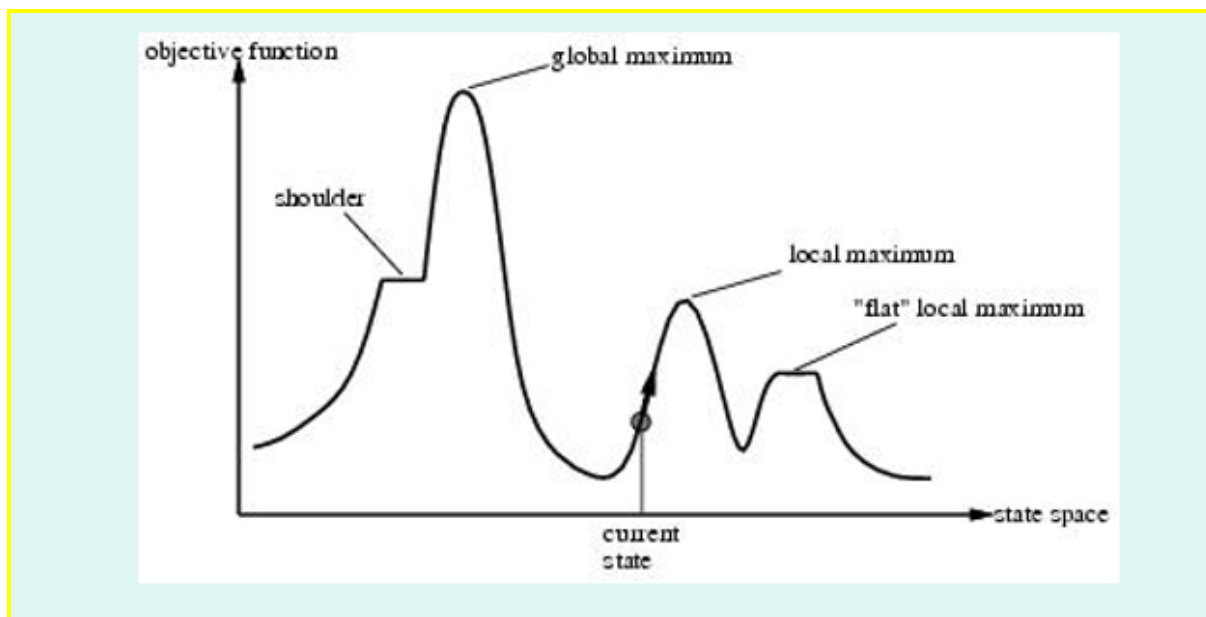
State space landscape: কল্পনা করো একটা ভূদৃশ্য — প্রতিটি location একটা state, উচ্চতা হলো সেই state-এর value (objective function)। লক্ষ্য হলো সর্বোচ্চ চূড়া (global maximum) খোঁজা।

৮. Hill-Climbing Search — Theory

Definition: Continuously moves in the direction of increasing value — সবসময় uphill যায়। Peak-এ পৌঁছালে থামে।

এটাকে **greedy local search** বলা হয় কারণ immediate neighbors ছাড়া কিছু দেখে না।

Algorithm: $current \leftarrow initial\ state \rightarrow loop: neighbor \leftarrow highest\text{-valued}\ successor \rightarrow$ যদি $neighbor\text{-এর}\ value \leq current\text{-এর}\ value$, তাহলে $return\ current\ (peak!)$ → নইলে $current \leftarrow neighbor$ ।



তিনটি বড় সমস্যা:

Local Maxima: এমন একটা peak যেখান থেকে আর উপরে যাওয়া যায় না, কিন্তু এটা global maximum নয়। Algorithm এখানে আটকে থাকে।

Plateau: সব neighbor-এর value সমান — flat area। Algorithm কোনদিকে যাবে বুঝতে পারে না।

Ridge: সরু পর্বতচূড়া যেখানে প্রতিটি দিকে যাওয়া কঠিন। Algorithm oscillate করে।

Statistics: Hill climbing 86% ক্ষেত্রে random 8-queens instance-এ local maximum-এ আটকে যায়।

Hill-climbing হলো একটি লোকাল সার্চ অ্যালগরিদম যা সবসময় উন্নতির দিকে (সবচেয়ে উঁচু চূড়ায় বা পাহাড়ে ওঠার মতো) যাওয়ার চেষ্টা করে। কিন্তু সাধারণ Hill-climbing অনেক সময় ভুল চূড়ায় (Local Maxima) আটকে যায়। এই সমস্যা সমাধানের জন্যই নিচের এই ৩টি ভ্যারিয়েশন তৈরি করা হয়েছে:

১. Stochastic Hill-climbing

এটি সাধারণ Hill-climbing-এর চেয়ে একটু আলাদা। সাধারণ পদ্ধতিতে আমরা চারপাশের সবচেয়ে ভালো (Best) অপশনটা বেছে নিই। কিন্তু Stochastic-এ আমরা একটু অন্যভাবে কাজ করি।

- মূল কাজ: চারপাশের সবগুলো ভালো বা "উঁচু" (Uphill) মুন্ডের মধ্য থেকে যেকোনো একটিকে র‍্যান্ডমলি (Randomly) বেছে নেওয়া হয়।
- মজার বিষয়: যে মুন্ডটি যত বেশি ভালো বা খাড়া (Steep), সেটি বেছে নেওয়ার সম্ভাবনা তত বেশি থাকে।
- কেন ব্যবহার করি? এটি সব সময় একই পথে না গিয়ে ভিন্ন ভিন্ন পথে চলার সুযোগ দেয়, যা অনেক সময় ভালো রেজাল্ট আনে।

২. First-choice Hill-climbing

নাম শুনেই বোঝা যাচ্ছে, এটি খুব একটা বাছবিচার করে না।

- মূল কাজ: এটি র‍্যান্ডমলি একের পর এক সাকসেসর (successor) তৈরি করতে থাকে এবং প্রথম যে অপশনটি বর্তমান অবস্থার চেয়ে একটু ভালো মনে হয়, সাথে সাথে সেদিকে চলে যায়।

- কখন ভালো? যখন আমাদের কাছে হাজার হাজার অপশন থাকে এবং সবগুলো চেক করার সময় থাকে না, তখন এটি খুব দ্রুত সিদ্ধান্ত নিতে সাহায্য করে।

৩. Random-restart Hill-climbing

এটি হলো সবচেয়ে শক্তিশালী ভ্যারিয়েশন। এর মূল মন্ত্র হলো— "একবার ব্যর্থ হলে বার বার চেষ্টা করো, তবে ভিন্ন জায়গা থেকে"।

- মূল কাজ: অ্যালগরিদমটি বারবার Hill-climbing চালায়। প্রতিবার সে একটি র‍্যান্ডমলি সিলেক্ট করা নতুন জায়গা (**Starting point**) থেকে যাত্রা শুরু করে।
- কেন সেরা? এটি **Local Maxima** (ছোট খাটো পাহাড়ের চূড়া) এ আটকে যাওয়া সমস্যার সমাধান করে। কারণ একবার যদি সে কোনো ভুল চূড়ায় আটকে যায়, তবে পরের বার সে অন্য এক জায়গা থেকে শুরু করে আসল হিমালয় বা Global Maxima খুঁজে পাওয়ার চেষ্টা করে।
- স্লোগান: "If at first you don't succeed, try, try again from a random location."

সহজ উদাহরণ:

ধরো তুমি ঘন কুয়াশার মধ্যে একটি পাহাড়ে উঠতে চাইছো:

- **Stochastic:** তুমি চারপাশের ২-৩টি উঁচু ঢাল দেখে লটারি করে যেকোনো একটিতে পা দিলে।
- **First-choice:** তুমি কুয়াশার মধ্যে হাতড়ে যদি কেই একটু উঁচুর আভাস পেলে, সেদিকেই হাঁটা শুরু করলে।
- **Random-restart:** তুমি এক পাহাড়ে উঠে দেখলে এটা আসল চূড়া না। তখন তুমি প্যারাসুটে করে অন্য এক অজানা জায়গায় গিয়ে নামলে এবং আবার ওঠা শুরু করলে, যতক্ষণ না সর্বোচ্চ চূড়ায় পৌঁছাও।

★ Comparison

Method	Key Idea	Advantage	Problem Solved
Stochastic HC	Random uphill selection	Avoid deterministic choice	Escapes tie situations in plateaus
First-Choice HC	Stop at first better neighbour	Fast	Reduces computation
Random-Restart HC	Multiple random starts	Avoid local maxima	Increases chance of global maximum

৯. Simulated Annealing — Theory

Hill climbing-এর সমস্যা হলো local maximum থেকে বের হতে পারে না। Simulated Annealing এটা solve করে **bad move (worse neighbor) accept** করার মাধ্যমে, কিন্তু সেই probability সময়ের সাথে কমতে থাকে।

Inspiration: Metallurgy-তে annealing — ধাতুকে গরম করে ধীরে ধীরে ঠান্ডা করা হয় যাতে atoms সবচেয়ে stable configuration-এ পৌঁছায়।

বলের **analogy:** একটা বল কল্পনা করো landscape-এর উপর — জোরে ঝাঁকালে (high temperature) সে যেকোনো জায়গায় যেতে পারে, ধীরে ধীরে কম ঝাঁকালে (temperature কমানো) সে global minimum-এ settle করে।

Core mechanism:

$$\Delta E = \text{VALUE}[\text{next}] - \text{VALUE}[\text{current}]$$

যদি $\Delta E > 0$ (next ভালো): সবসময় accept করো।

যদি $\Delta E < 0$ (next খারাপ): $e^{(\Delta E/T)}$ probability-তে accept করো।

এখানে T হলো **temperature**। T বেশি থাকলে $e^{(\Delta E/T)}$ এর মান বেশি হয় → খারাপ move-ও accept হওয়ার সম্ভাবনা বেশি (exploration)। T কমলে এই probability কমে → আস্তে আস্তে Hill Climbing-এর মতো আচরণ করে।

Schedule: কতটা দ্রুত T কমানো হবে তার plan। যদি T যথেষ্ট slowly কমে, তাহলে theoretically global optimum পাওয়ার guarantee আছে।

Application: VLSI layout, airline scheduling।

১০. Local Beam Search — Theory

Hill climbing-এ শুধু একটা state track করা হয়। Local Beam Search-এ **k** টা **state** track করা হয়।

শুরু হয় k টা random state দিয়ে। প্রতি step-এ এই k টা state-এর সব successor generate হয়। যদি কোনো successor goal হয় — শেষ। নইলে সব successor থেকে সেরা k টা বেছে নিয়ে continue।

Random-restart HC থেকে পার্থক্য: Random restart-এ k টা independent thread চলে। Beam Search-এ k টা thread তথ্য **share** করে — ভালো state-এর successor পেলে সবাই সেদিকে move করে।

সমস্যা: Diversity কমে যেতে পারে — সব k state একই জায়গায় converge করে। **Stochastic Beam Search** এটা solve করে — top-k না বেছে, fitness-proportional random selection করে।

Feature	Local Beam Search	Genetic Algorithm
Multiple states	Yes (k current best)	Yes (population)
Successor generation	From current neighbors	From crossover of parents
Selection	Top-k successors	Fitness-based selection of parents
Diversity mechanism	Limited (select best k)	Yes — via mutation + crossover
Information sharing	No (each state evolves alone)	Yes (via recombination between solutions)
Inspired by	Heuristic search	Evolution / biology

১১. Genetic Algorithms — সবচেয়ে বিস্তারিত Theory

১১.১ GA কী এবং কোথা থেকে এলো?

GA হলো **Local Beam Search**-এর একটি **variant** যেখানে **sexual recombination** আছে। অর্থাৎ, দুটো state "মিলিত" হয়ে নতুন state তৈরি করে।

এটা অনুপ্রাণিত **biological evolution** থেকে — প্রকৃতিতে যোগ্য প্রাণী টিকে থাকে এবং reproduce করে, তাদের সন্তান আরও ভালো হয়।

১১.২ মূল Terminology

Population: k টা randomly generated state-এর সমষ্টি, প্রতিটিকে **individual** বলে।

Chromosome / Individual: প্রতিটি state-কে একটা finite alphabet-এর string হিসেবে represent করা হয়। 8-queens-এ ৮টি digit (1-8), যেখানে প্রতিটি digit সেই column-এ queen কোন row-এ আছে।

Gene: Chromosome-এর প্রতিটি element।

Fitness Function: প্রতিটি individual কতটা ভালো তা measure করে। 8-queens-এ: non-attacking queen pairs-এর সংখ্যা (maximum = 28)। বেশি **fitness** = বেশি ভালো।

Selection: Fitness অনুযায়ী reproduce করার জন্য parent বেছে নেওয়া। বেশি fit individual-এর বেশি chance।

Crossover: দুটো parent থেকে child তৈরি করা — একটা crossover point বেছে নিয়ে প্রথম parent থেকে শুরু এবং দ্বিতীয় parent থেকে বাকি অংশ নেওয়া হয়।

Mutation: ছোট probability-তে একটা random gene পরিবর্তন করা। 8-queens-এ: একটা queen random square-এ move করা।

১১.৩ GA কীভাবে কাজ করে — Step by Step Theory

Step 1 — Initialization: k টা random individual তৈরি করো। এটাই initial population।

Step 2 — Fitness Evaluation: প্রতিটি individual-এর fitness score বের করো। 8-queens উদাহরণে চারটি individual-এর score হলো 24, 23, 20, 11। Percentage-এ: 31%, 29%, 26%, 14%।

Step 3 — Selection: Fitness-proportional probability-তে parent pair বাছাই করো। যার fitness বেশি, তার reproduce করার chance বেশি। একই individual একাধিকবার select হতে পারে, কেউ একবারও নাও হতে পারে।

Step 4 — Crossover: প্রতি pair-এর জন্য একটা random crossover point বেছে নাও। প্রথম child পায় parent 1-এর শুরু + parent 2-এর শেষ। দ্বিতীয় child পায় parent 2-এর শুরু + parent 1-এর শেষ।

উদাহরণ: Parent 1 = 32748552, Parent 2 = 24748552, crossover point = 3। Child 1 = **327** | 48552 → 32748552 Child 2 = **247** | 48552 → 24748552

Step 5 — Mutation: ছোট independent probability-তে প্রতিটি position-এ random change। এটা diversity নিশ্চিত করে এবং local optima থেকে বের হতে সাহায্য করে।

Step 6 — New Generation: Children মিলে নতুন population তৈরি হয়। আবার Step 2 থেকে repeat।

Termination: কোনো individual যথেষ্ট fit হলে, অথবা নির্দিষ্ট সময় পার হলে, সেরা individual return করো।

১১.৪ GA কেন কাজ করে — Schema Theory

এটা খুব important theoretical part।

Schema হলো chromosome-এর একটা pattern, যেখানে কিছু position fixed এবং কিছু wildcard। যেমন "3*7*****" মানে 1ম position 3, 3য় position 7, বাকি যেকোনো।

Schema theory বলে: যে **schema**-গুলোর **fitness** বেশি, সেগুলো পরবর্তী **generation**-এ বেশি **represent** হবে। এভাবে ভালো "building blocks" combine হয়ে আরও ভালো solution তৈরি হয়।

Early search-এ: Population diverse থাকে → crossover বড় পরিবর্তন আনে → broad exploration।

Late search-এ: সবাই similar → crossover ছোট পরিবর্তন আনে → fine-tuning/exploitation।

এটা Simulated Annealing-এর temperature schedule-এর analogous।

১১.৫ GA vs Local Beam Search — Theoretical Difference

বিষয়	Local Beam Search	Genetic Algorithm
Multiple states	হ্যাঁ (k current best)	হ্যাঁ (population)
Successor generation	Current neighbors থেকে	Parents-এর crossover থেকে
Selection	Top-k successors	Fitness-based
Diversity	সীমিত	Mutation + crossover
Information sharing	নেই (each state alone)	আছে (recombination)
Inspired by	Heuristic search	Biological evolution

১১.৬ Crossover কেন গুরুত্বপূর্ণ?

যখন দুটো parent অনেক আলাদা, তখন crossover অনেক দূরের state-এ পৌঁছাতে পারে। এটা large state space efficiently explore করতে দেয়। Mutation শুধু ছোট local change করে, কিন্তু crossover একসাথে বড় jump করতে পারে।

১২. সব Algorithm-এর Theoretical Comparison

Greedy BFS: $f(n) = h(n)$ only। Complete নয়, Optimal নয়। দ্রুত কিন্তু অনির্ভরযোগ্য।

A Search: $f(n) = g(n) + h(n)$ । Complete ✓, Optimal ✓ (যদি h admissible হয়)। সেরা algorithm path-finding-এ, কিন্তু memory সমস্যা।

Hill Climbing: শুধু current state + neighbors। Memory $O(1)$ । Local maxima-তে আটকে যায়।

Simulated Annealing: Bad move accept করে temperature অনুযায়ী। Slowly cool করলে theoretically optimal।

Local Beam Search: k states parallelly track করে। Information share করে কিন্তু diversity কমে যেতে পারে।

Genetic Algorithm: Population + crossover + mutation। বিবর্তন-অনুপ্রাণিত। Complex optimization-এ সবচেয়ে শক্তিশালী।

১৩. সবচেয়ে গুরুত্বপূর্ণ যে তিনটা জিনিস মনে রাখবে

A এর জন্য: $f(n) = g(n) + h(n)$, $h(n)$ must be admissible অর্থাৎ $h(n) \leq h^*(n)$, তাহলেই complete এবং optimal।

Genetic Algorithm-এর জন্য: Population → Fitness Evaluation → Selection → Crossover → Mutation → New Generation → Repeat। এটা Local Beam Search-এর variant কারণ উভয়ই k states রাখে, কিন্তু GA-তে crossover দিয়ে information combine হয়।

Hill Climbing-এর জন্য: Greedy local search, 86% ক্ষেত্রে local maximum-এ আটকায়, তিনটা variant (Stochastic, First-Choice, Random-Restart)।

Time & Space Complexity — সম্পূর্ণ Theory

প্রথমে বুঝো: Complexity কী মাপে?

Time Complexity মাপে — algorithm কতটা কাজ করে (কতটা node explore করে)।

Space Complexity মাপে — algorithm কতটা মেমোরি ব্যবহার করে (একসাথে কতটা node store করে রাখে)।

দুটো parameter দিয়ে express করা হয়:

- **b** = branching factor (প্রতিটি node থেকে গড়ে কতটা child)
- **d** = depth of shallowest goal (সবচেয়ে কাছের goal কত গভীরে)
- **m** = maximum depth of search tree
- **k** = number of states (local/beam search-এ)

১. Greedy Best-First Search

Time Complexity: $O(b^m)$

Worst case-এ পুরো search tree generate হতে পারে। m হলো maximum depth। কারণ হলো — Greedy কোনো guarantee রাখে না যে সে সঠিক path-এ আছে। ভুল দিকে গেলে পুরো subtree explore করে ফেলে।

কিন্তু একটা ভালো heuristic হলে এই complexity **dramatically** কমে যায়। এটাই greedy-র একমাত্র সুবিধা।

Space Complexity: $O(b^m)$

সব generated node memory-তে রাখে (fringe/priority queue-এ)। Worst case-এ এটাও $O(b^m)$ ।

Summary

Complete? না — loop হলে আটকে যায়, repeated state check না করলে।

Optimal? না — শুধু $h(n)$ দেখে, $g(n)$ ignore করে।

Greedy সবচেয়ে fast হতে পারে ভালো heuristic-এ, কিন্তু কোনো guarantee নেই।

২. A* Search

Time Complexity: $O(b^d)$

Theoretically — যদি heuristic perfect হয় ($h = h^*$), তাহলে A* শুধু optimal path-এর nodes explore করে $\rightarrow$ linear time।

কিন্তু practical case-এ heuristic perfect না, তাই worst case **exponential** — $O(b^d)$ বা আরও বেশি। Specifically, যদি heuristic error থাকে তাহলে expanded nodes exponentially বাড়ে।

Crucial theoretical point: A* হলো **optimally efficient** — অর্থাৎ কোনো optimal algorithm, একই $h(n)$ দিয়ে, A* থেকে কম node expand করতে পারবে না। এটা prove করা হয়েছে।

Space Complexity: $O(b^d)$

এটাই A*-এর সবচেয়ে বড় দুর্বলতা।

A* সব generated node memory-তে রাখে — কারণ যেকোনো node পরে revisit লাগতে পারে। Depth d পর্যন্ত সব node store হয়।

গুরুত্বপূর্ণ: A* সাধারণত **time**-এর আগে **space** শেষ করে ফেলে। Real problem-এ এটা impractical হয়ে যায়।

এই কারণেই **IDA*** (Iterative Deepening A*) এবং **RBFS** (Recursive Best-First Search) তৈরি হয়েছে — এগুলো কম memory ব্যবহার করে।

Summary

Complete? হ্যাঁ (যদি finite nodes থাকে $f < f(G)$)।

Optimal? হ্যাঁ (যদি $h(n)$ admissible হয়)।

৩. Hill Climbing

Time Complexity: $O(\infty)$ worst case, কিন্তু practically fast

Hill climbing-এর কোনো formal time complexity নেই সেভাবে, কারণ এটা **complete** নয়। Local maximum-এ আটকে গেলে কখনো goal নাও পেতে পারে।

Practical case-এ প্রতিটি step-এ শুধু current node-এর neighbors evaluate হয় → প্রতি iteration $O(b)$ । কিন্তু কতটা iteration লাগবে তার guarantee নেই।

Space Complexity: $O(1)$ ← এটাই Hill Climbing-এর সবচেয়ে বড় সুবিধা

শুধু দুটো জিনিস store করে রাখে:

- Current state
- Current state-এর value (evaluation)

কোনো search tree নেই, কোনো fringe নেই, আগের কোনো state মনে রাখে না। এই কারণে বিশাল state space-এও কাজ করতে পারে।

Summary

Complete? না।

Optimal? না।

Memory $O(1)$ — এটাই Hill Climbing-কে আলাদা করে।

8. Simulated Annealing

Time Complexity: নির্ভর করে **cooling schedule**-এর উপর

Theoretical guarantee: যদি temperature **logarithmically slowly** কমানো হয়, তাহলে global optimum পাওয়ার probability → 1। কিন্তু এতে infinitely অনেক সময় লাগতে পারে।

Practical case: Polynomial বা faster cooling schedule ব্যবহার করা হয়। তখন time কমে কিন্তু optimality-র guarantee থাকে না।

Formal time complexity express করা কঠিন কারণ এটা stochastic — প্রতিবার ভিন্ন হতে পারে।

Space Complexity: $O(1)$

Hill climbing-এর মতোই। শুধু রাখে:

- Current state
- Next (randomly selected) state
- Temperature T

কোনো history বা tree নেই।

Summary

Complete? **Probabilistically** হ্যাঁ — যথেষ্ট সময় দিলে।

Optimal? **Probabilistically** হ্যাঁ — slow cooling হলে।

৫. Local Beam Search

Time Complexity: $O(b \times k)$ per iteration

প্রতি iteration-এ k states-এর প্রতিটির সব b successor evaluate হয়। Total = $k \times b$ per step। কতটা iteration লাগবে তার guarantee নেই।

Worst case overall এখনো exponential হতে পারে।

Space Complexity: $O(k \times b)$

k টা current state + তাদের সব successor ($k \times b$) একসাথে memory-তে রাখতে হয়। Hill climbing-এর $O(1)$ থেকে বেশি, কিন্তু A^* -এর $O(b^d)$ থেকে অনেক কম।

Summary

Complete? না।

Optimal? না।

k বাড়লে ভালো solution পাওয়ার সম্ভাবনা বাড়ে কিন্তু memory বাড়ে।

৬. Genetic Algorithm

Time Complexity: $O(g \times k \times L)$

এখানে:

- g = number of generations (কতটা iteration)
- k = population size
- L = length of chromosome (individual-এর size)

প্রতিটি generation-এ k individual-এর fitness evaluate হয়, crossover ও mutation হয়। কতটা generation লাগবে তার কোনো guarantee নেই — এটা problem-এর উপর নির্ভরশীল।

Theoretical note: GA convergence-এর exact time bound প্রমাণ করা কঠিন কারণ এটা stochastic। Schema theorem বলে ভালো schemas exponentially বাড়ে, কিন্তু overall convergence time problem-specific।

Space Complexity: $O(k \times L)$

k টা individual, প্রতিটির length L । শুধু current population store করতে হয়। আগের generation বাদ দেওয়া যায়।

Hill climbing-এর $O(1)$ থেকে বেশি, কিন্তু A^* -এর $O(b^d)$ থেকে অনেক কম এবং controllable।

Summary

Complete? **Probabilistically** হ্যাঁ — যথেষ্ট large population ও generation দিলে।

Optimal? **Probabilistically** হ্যাঁ।

৭. Greedy Search Algorithm (General)

এখানে "Greedy Algorithm" বলতে general greedy approach বোঝানো হচ্ছে — যেটা প্রতিটি step-এ locally সেরা choice নেয়।

Time Complexity: $O(n \log n)$ থেকে $O(n^2)$ — problem-এ নির্ভর করে

সাধারণত greedy algorithm-এ sorting দরকার হয় $\rightarrow O(n \log n)$ । তারপর প্রতিটি element একবার process $\rightarrow O(n)$ । মোট: $O(n \log n)$ ।

কিছু problem-এ (যেমন Activity Selection, Huffman Coding) greedy $O(n \log n)$ -এ optimal solution দেয়।

Space Complexity: $O(1)$ থেকে $O(n)$

Extra data structure না লাগলে $O(1)$ । Output store করতে হলে $O(n)$ ।

Theoretical Note

Greedy সবসময় optimal নয়। শুধু সেই problem-এ optimal যেখানে **greedy choice property** এবং **optimal substructure** থাকে। যেমন: Minimum Spanning Tree (Kruskal's), Shortest Path (Dijkstra's), Huffman Encoding।

৮. Minimax Algorithm

Background Theory

Minimax হলো **two-player zero-sum game**-এর জন্য — যেমন Chess, Tic-tac-toe। একজন player **maximize** করতে চায় (MAX player), অপরজন **minimize** করতে চায় (MIN player)। প্রতিটি node-এ alternately MAX ও MIN খেলে।

মূল ধারণা: MAX player ধরে নেয় MIN player সবসময় তার জন্য সেরা (MAX-এর জন্য worst) move করবে। তাই MAX এমন move বেছে নেয় যেটা MIN-এর সেরা response-এও ভালো outcome দেয়।

Time Complexity: $O(b^m)$

পুরো game tree generate করতে হয় depth m পর্যন্ত। প্রতিটি node-এ b টা possible move। তাই total nodes = b^m ।

Chess-এ $b \approx 35$, $m \approx 100 \rightarrow 35^{100}$ যা computationally impossible। তাই real game-এ depth limit দেওয়া হয়।

Space Complexity: $O(b \times m)$

Minimax **depth-first** explore করে। তাই একসাথে শুধু current path store করতে হয়। Path-এর depth m , প্রতিটি level-এ b টা node। তাই $O(b \times m)$ বা simply $O(bm)$ ।

এটা time complexity-র তুলনায় অনেক কম — এটাই DFS-based approach-এর সুবিধা।

Summary

Complete? হ্যাঁ — যদি tree finite হয়।

Optimal? হ্যাঁ — যদি opponent-ও optimally খেলে।

৯. Alpha-Beta Pruning

Background Theory

Minimax-এর সমস্যা হলো $O(b^m)$ time — পুরো tree দেখতে হয়। Alpha-Beta Pruning এটা solve করে **irrelevant branches prune** করে — যে branches-এর result কখনো use হবে না সেগুলো দেখেই না।

Alpha (α): MAX player এখন পর্যন্ত যে সেরা value guarantee করতে পারে। শুরুতে $-\infty$ ।

Beta (β): MIN player এখন পর্যন্ত যে সেরা value guarantee করতে পারে। শুরুতে $+\infty$ ।

Pruning condition: যদি কোনো node-এ $\alpha \geq \beta$ হয়, তাহলে সেই subtree আর দেখার দরকার নেই।

কেন? কারণ MIN player ইতিমধ্যে এর চেয়ে ভালো option পেয়ে গেছে $\rightarrow$ MAX player এই path কখনো choose করবে না।

Time Complexity

Best case: $O(b^{(m/2)})$

এটা অনেক important। যদি moves সবসময় best-first order-এ থাকে (অর্থাৎ সেরা move আগে explore হয়), তাহলে Alpha-Beta, Minimax-এর same depth-এ দ্বিগুণ গভীরতা দেখতে পারে।

সহজ কথায়: Minimax যেখানে depth 6 দেখতে পারে, Alpha-Beta সেখানে depth 12 দেখতে পারে — same time-এ।

Worst case: $O(b^m)$

যদি moves সবসময় worst-first order-এ থাকে — কোনো pruning হয় না → Minimax-এর মতোই।

Average/Random case: $O(b^{(3m/4)})$

Random ordering-এ, practically Alpha-Beta $O(b^{(3m/4)})$ এর কাছাকাছি।

Space Complexity: $O(bm)$

Minimax-এর মতোই — DFS approach, তাই শুধু current path store করে। Pruning space-এ কোনো পরিবর্তন আনে না।

Crucial Theoretical Point

Alpha-Beta Pruning **Minimax**-এর মতোই **result** দেয় — same optimal move return করে। Pruning শুধু unnecessary nodes skip করে, কিন্তু decision পরিবর্তন করে না। এটা একটা **exact algorithm**, approximation নয়।

Move ordering-এর গুরুত্ব: সেরা move আগে explore করলে pruning বেশি হয় → best case $O(b^{(m/2)})$ এর কাছে যায়। এই কারণে real chess engines move ordering-এ অনেক সময় দেয়।

Summary

Complete? হ্যাঁ (finite tree হলে)।

Optimal? হ্যাঁ (Minimax-এর সমান result)।

সব একসাথে — Quick Reference

Algorithm	Time Complexity	Space Complexity	Complete	Optimal
Greedy BFS	$O(b^m)$	$O(b^m)$	✗	✗
A* Search	$O(b^d)$	$O(b^d)$	✓	✓*
Hill Climbing	No guarantee	$O(1)$	✗	✗
Simulated Annealing	Schedule-dependent	$O(1)$	Prob. ✓	Prob. ✓
Local Beam Search	$O(k \times b)$ per step	$O(k \times b)$	✗	✗

Genetic Algorithm	$O(g \times k \times L)$	$O(k \times L)$	Prob.	Prob.
General Greedy	$O(n \log n)$	$O(1) - O(n)$	Problem-dependent	Problem-dependent
Minimax	$O(b^m)$	$O(bm)$		
Alpha-Beta Pruning	$O(b^{(m/2)})$ best	$O(bm)$		

A optimal যখন $h(n)$ admissible।

সবচেয়ে গুরুত্বপূর্ণ তুলনামূলক Points

A vs Greedy BFS: দুটোই $O(b^m)$ worst case time, কিন্তু A* optimal আর Greedy নয়। A*-এর memory সমস্যা আছে।

Hill Climbing vs A: Hill Climbing $O(1)$ memory — অনেক practical। কিন্তু optimal বা complete নয়। A* optimal কিন্তু memory-intensive।

Minimax vs Alpha-Beta: Same result, same space। কিন্তু Alpha-Beta best case-এ দ্বিগুণ গভীরতা দেখতে পারে — এটাই পার্থক্য।

GA vs Hill Climbing: GA বেশি memory নেয় $O(k \times L)$ কিন্তু local optima থেকে বের হতে পারে। Hill Climbing $O(1)$ কিন্তু আটকে যায়।